SMART BANKING
DA 3

Introduction

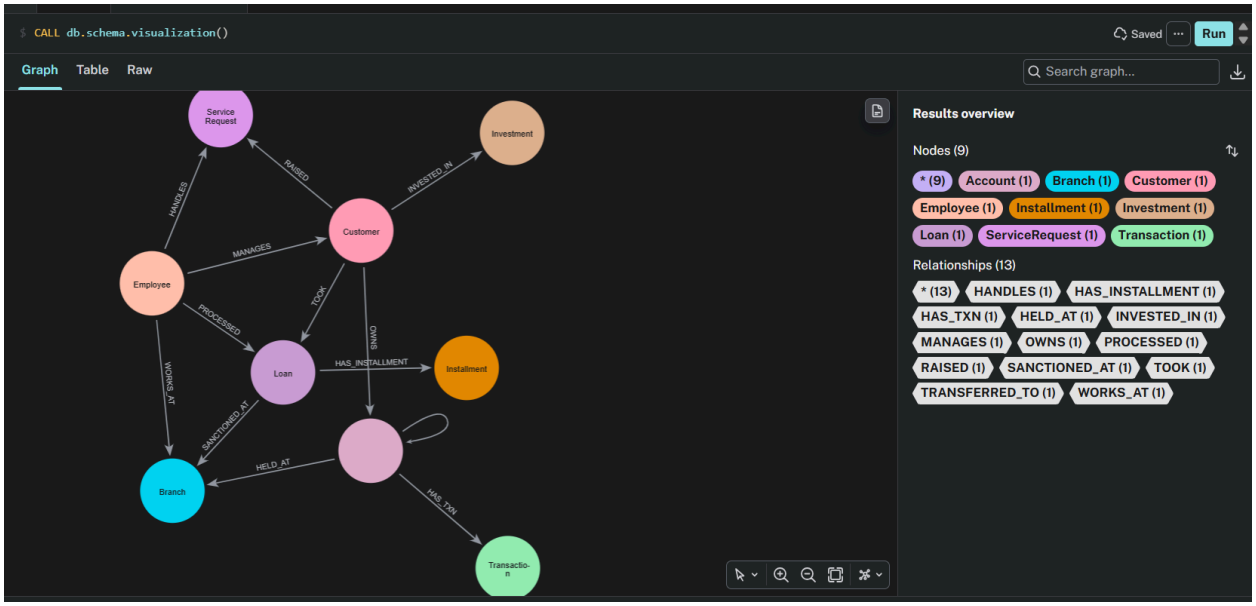
Smart Banking is a banking application built on a graph database. Customers, accounts, branches, employees, loans, investments and service requests are stored as nodes in Neo4j Aura, and the links between them (a customer owns an account, an account transfers money to another account) are stored as relationships. A Streamlit front end lets a user look up customers, move money, view loans and raise service requests. Neo4j suits banking because the key questions are about movement of money across accounts, such as where it went next and whether it returned to its source. These are path queries, which a graph handles in one short pattern but a relational database handles with repeated joins.

Technologies

Neo4j Aura (cloud graph database), Cypher query language, Python, Streamlit (front end), Neo4j Python driver.

Data model

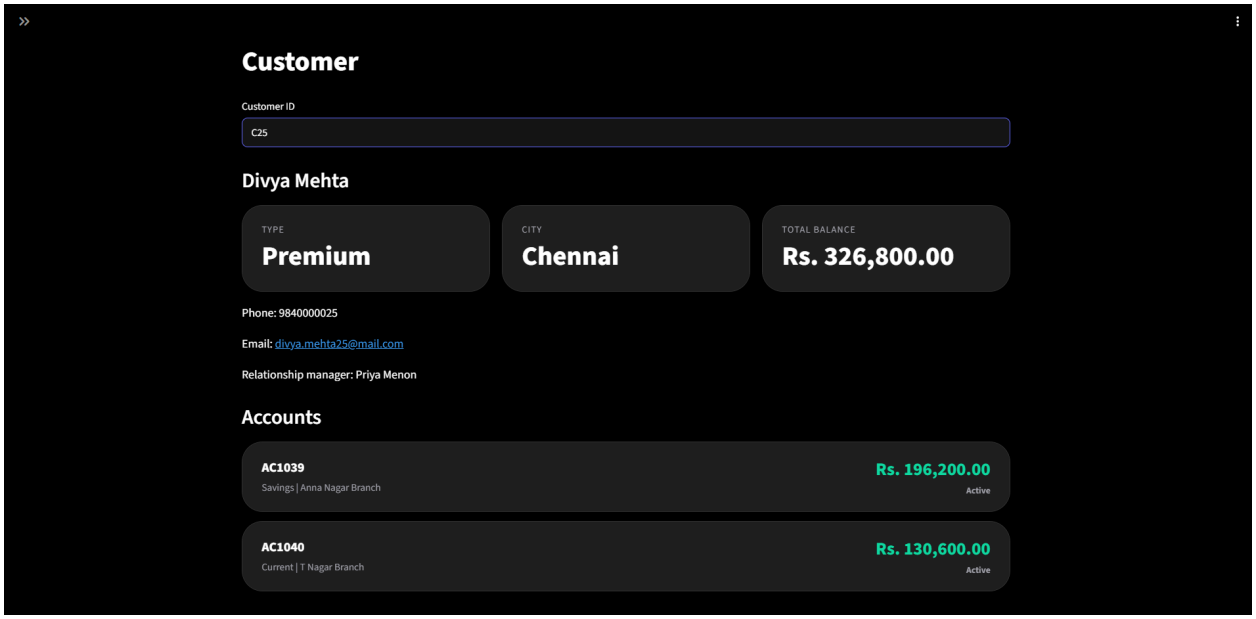
The graph has 9 node types (Customer, Account, Branch, Employee, Loan, Installment, Investment, Transaction, ServiceRequest) and 13 relationship types, such as OWNS, HELD_AT, TRANSFERRED_TO, TOOK, RAISED and HANDLES. Transfers between accounts are stored as relationships so they can be followed as paths.



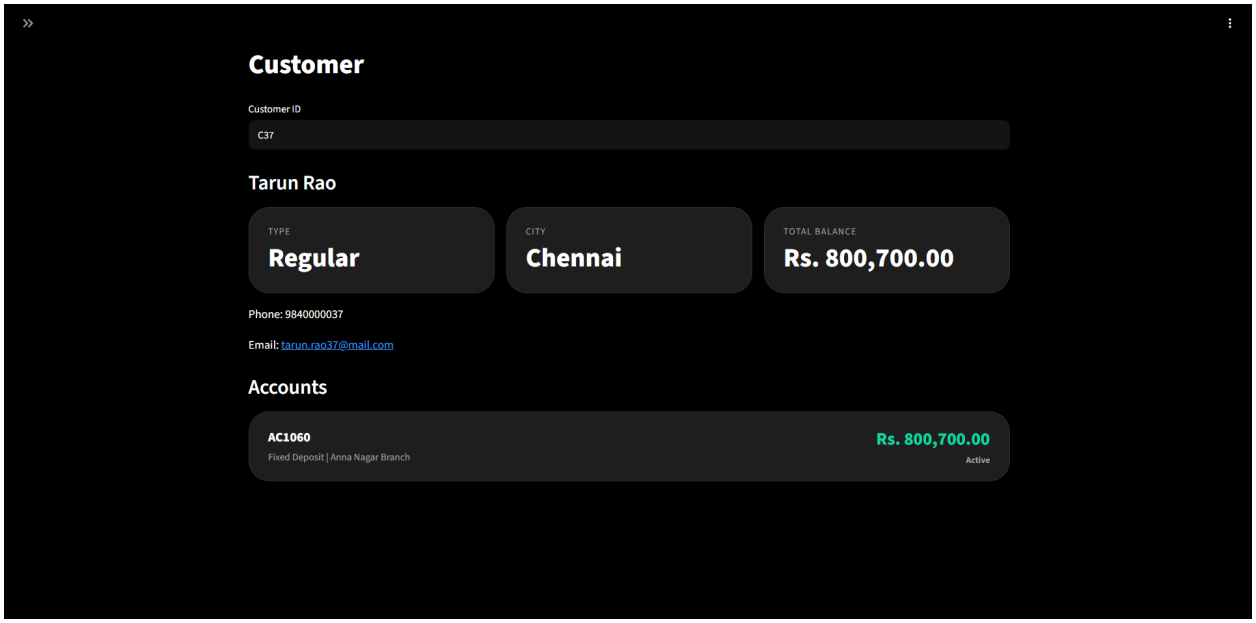
Application screens

Customer:

Customer C25 (Premium) with two accounts. The total balance is the sum of the account balances.



Checking details of Customer C37.



Raising a service request:

Customer C01 raises an Address Update request. It appears as Pending and is assigned to Karthik Suresh, a Customer Service Executive.

Customer

Request SR19ADCB28 raised

Customer ID

C01

Ishita Sharma

TYPE	CITY	TOTAL BALANCE
Premium	Chennai	Rs. 163,200.00

Phone: 9840011111

Email: ishita.sharma@mail.com

Relationship manager: Priya Menon

Accounts

AC1001
Savings | Anna Nagar Branch
Rs. 11,500.00
Active

AC1002
Current | Anna Nagar Branch
Rs. 151,700.00
Active

Service requests

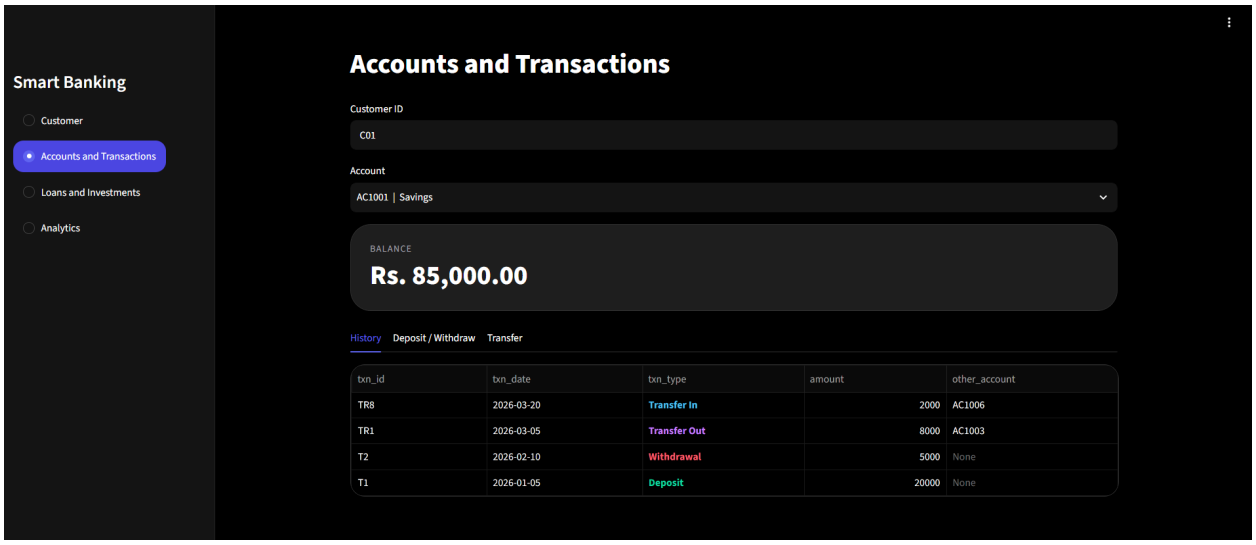
Cheque Book
SR03 | 2026-01-25 | Request new cheque book
Resolved
Handled by Karthik Suresh

Address Update
SR19ADCB28 | 2026-10-22 | need to update address for C01
Pending
Handled by Karthik Suresh

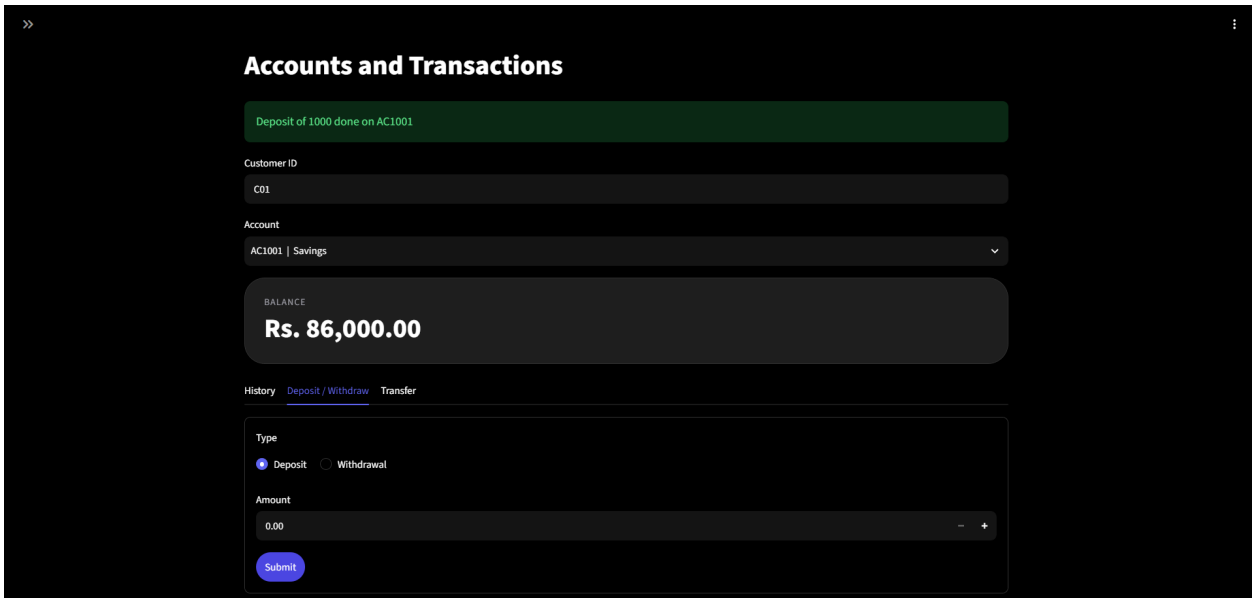
› Raise a new request

Accounts and Transactions:

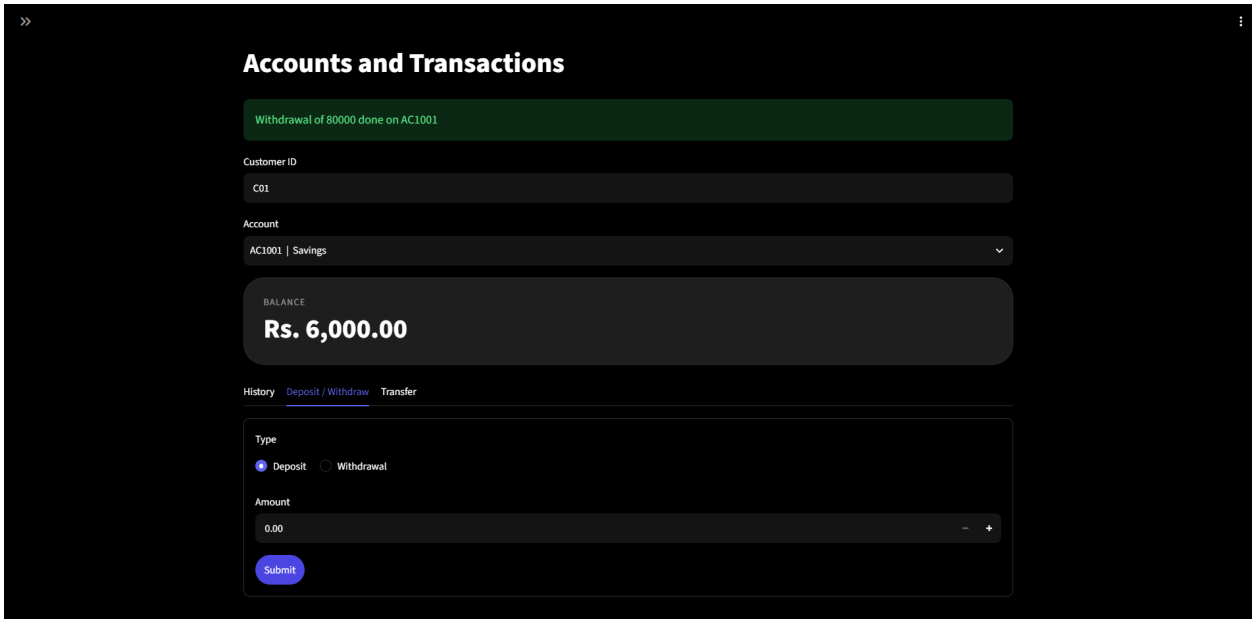
History of Customer C01.



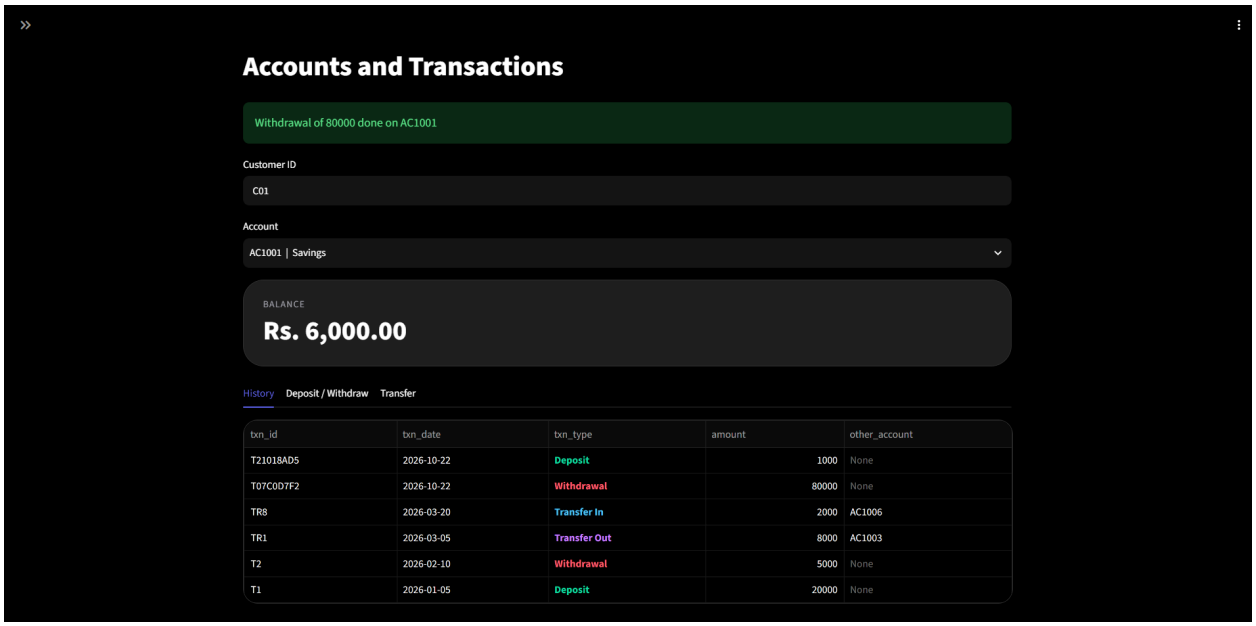
Depositing Rs. 1,000 to AC1001 raises the balance from Rs. 85,000 to Rs. 86,000.



Withdrawing Rs. 80,000 leaves a balance of Rs. 6,000.

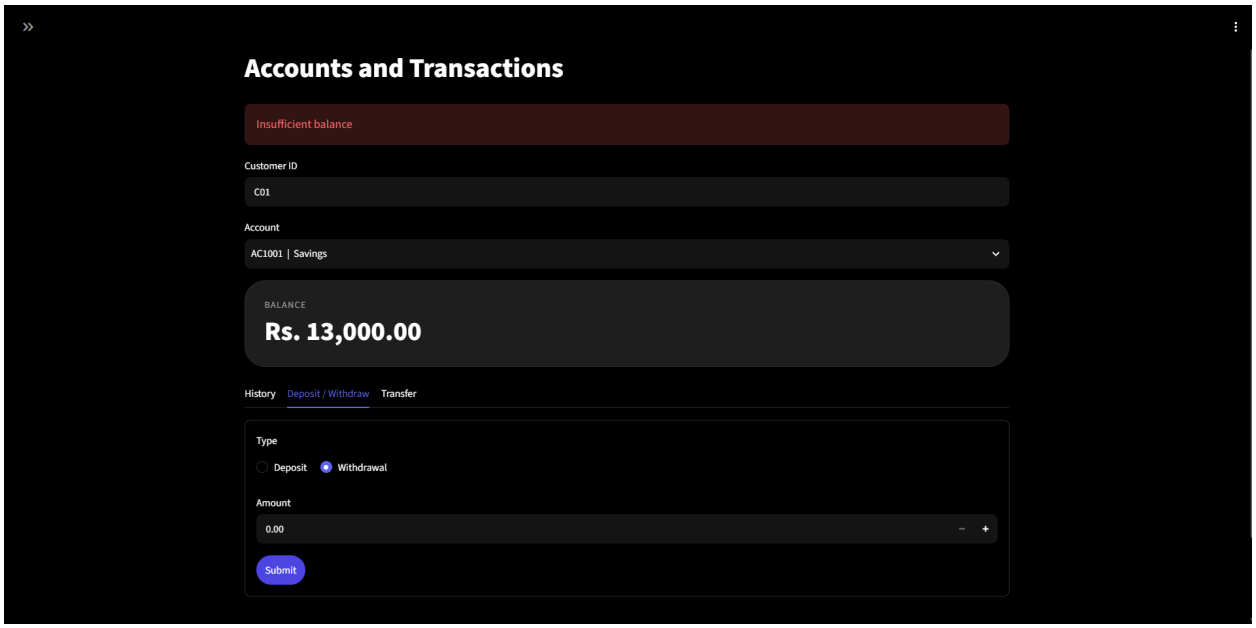


History of recent transactions.



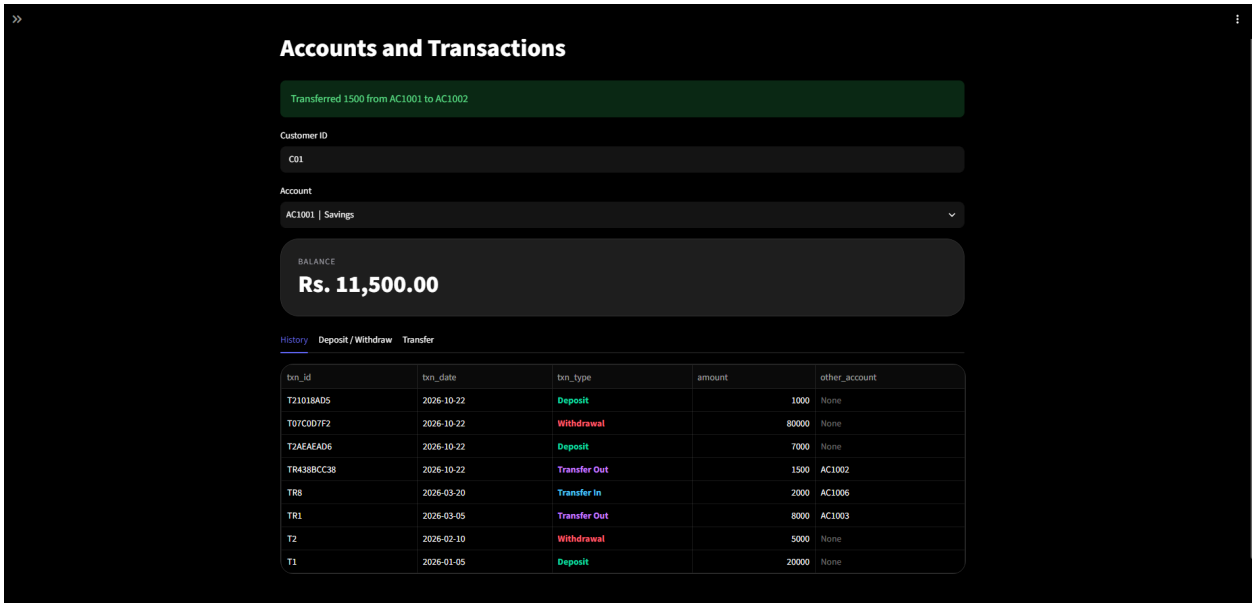
Withdrawing 14,000 (insufficient balance)

After a further deposit of Rs. 7,000 the balance is Rs. 13,000. A withdrawal larger than the balance is rejected.



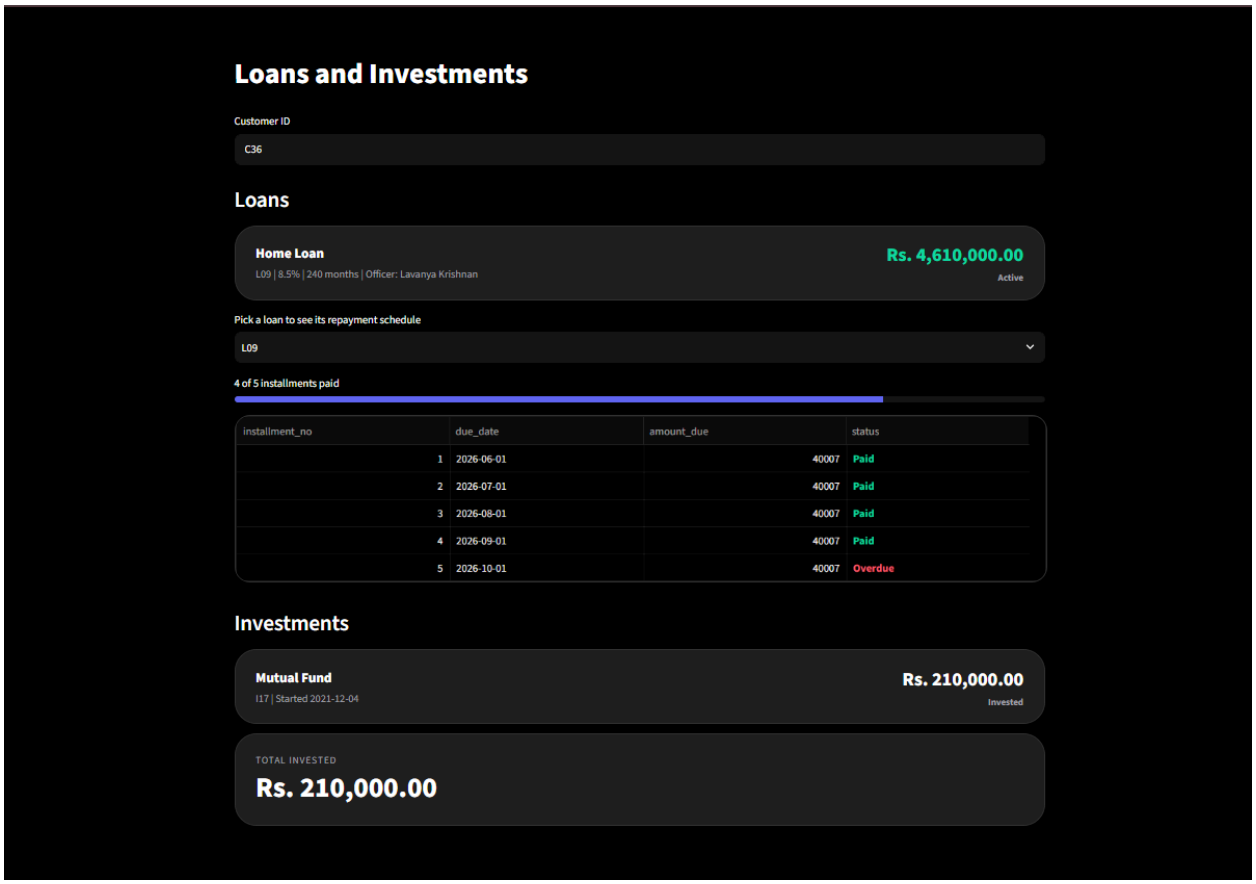
Transferring money from one account AC1001 to another AC1002.

Rs. 1,500 moves from AC1001 to AC1002. The balance becomes Rs. 11,500 and the transfer appears in the history.

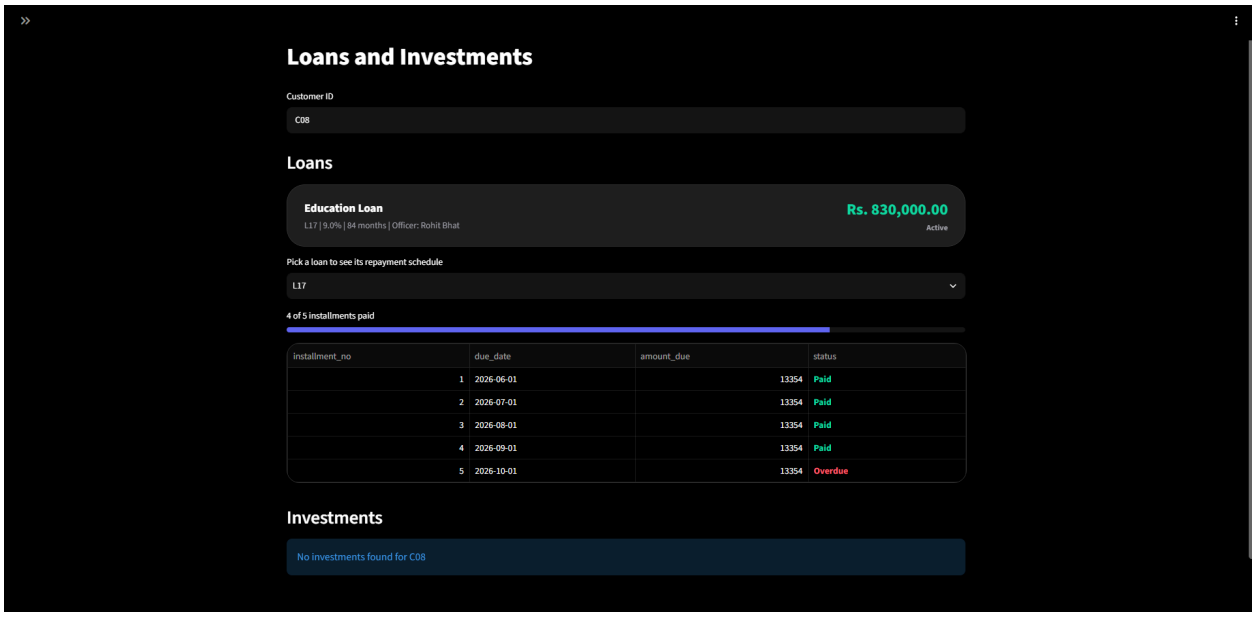


Loans and Investments:

Customer C36’s home loan with its repayment schedule. Four of five installments are paid and one is overdue.

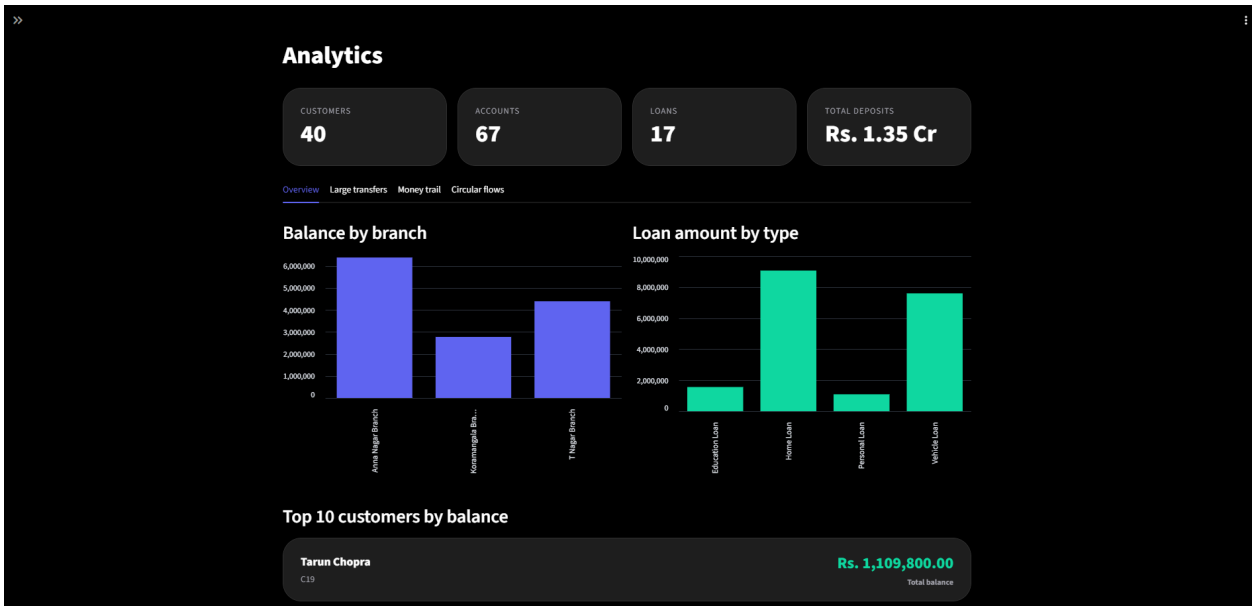


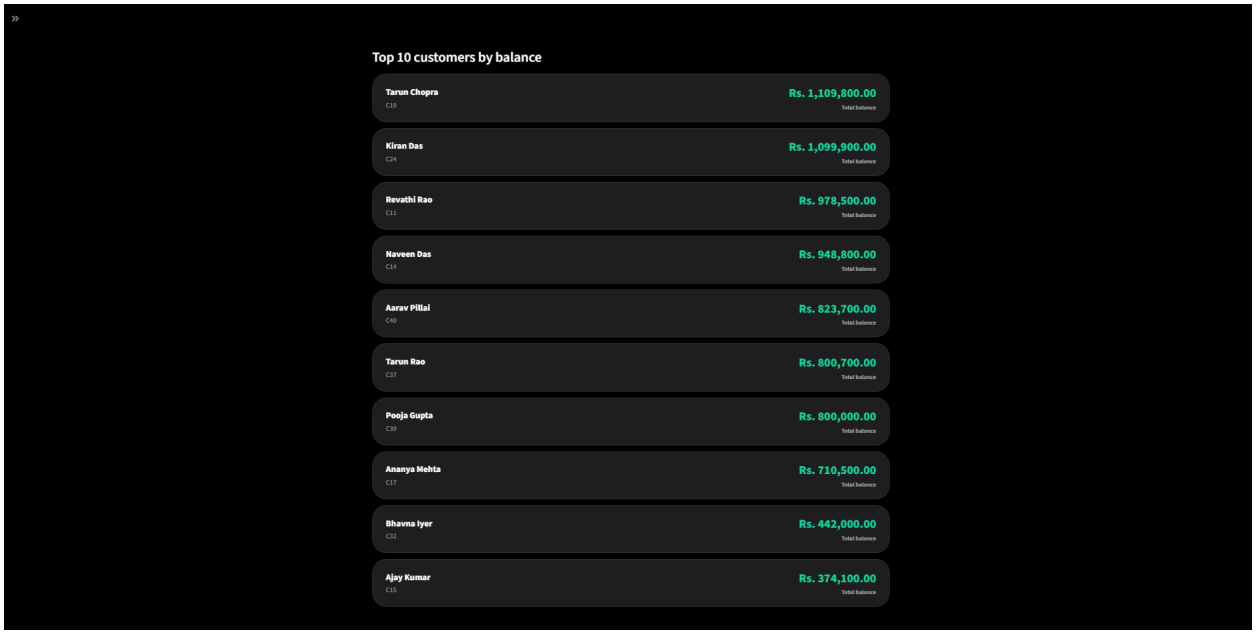
Customer C08 has an education loan and no investments.



Analytics:

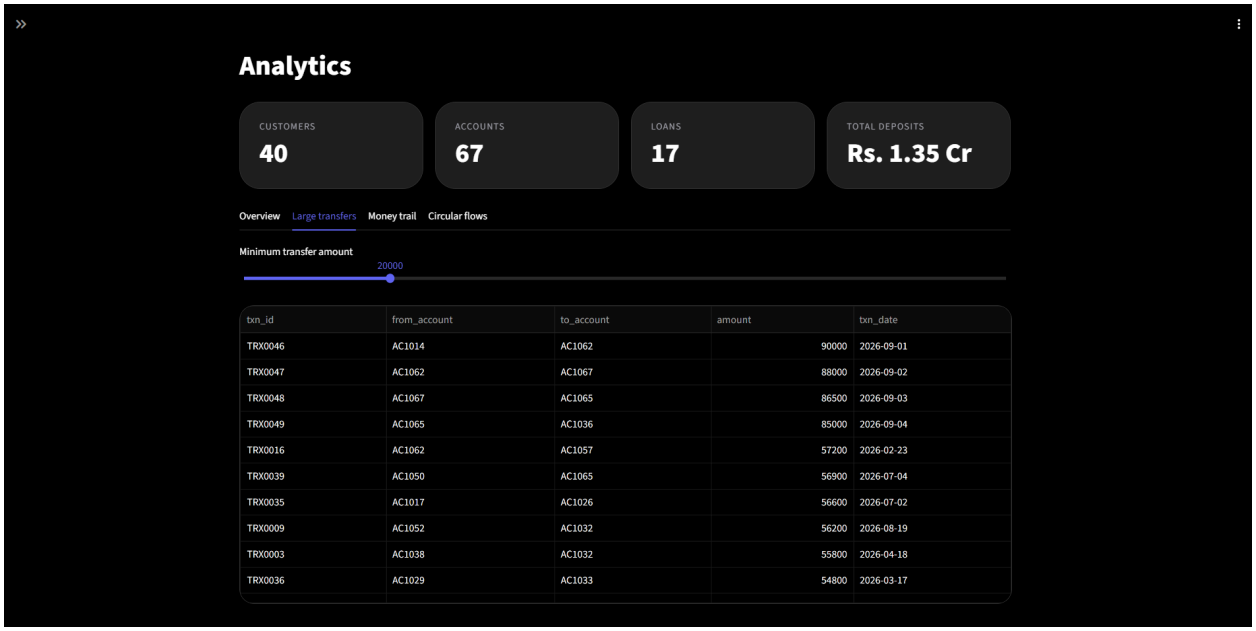
Balance by branch, loan amount by type and the top 10 customers by balance.





Large transfers:

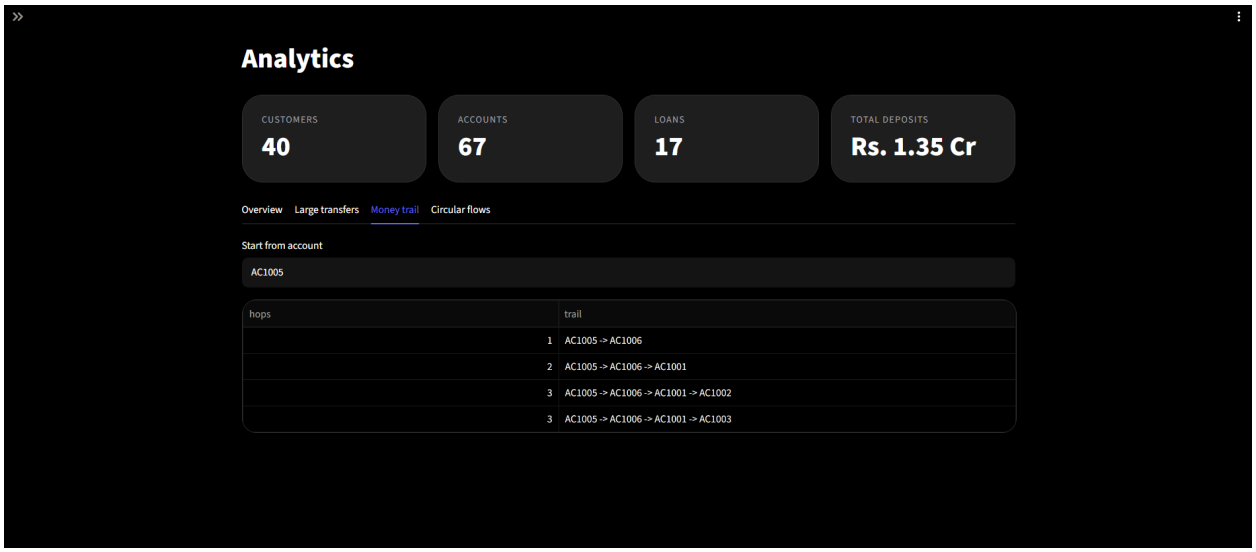
Transfers above Rs. 20,000. TRX0046 to TRX0049 form a chain of 90,000, 88,000, 86,500 and 85,000 over four days.



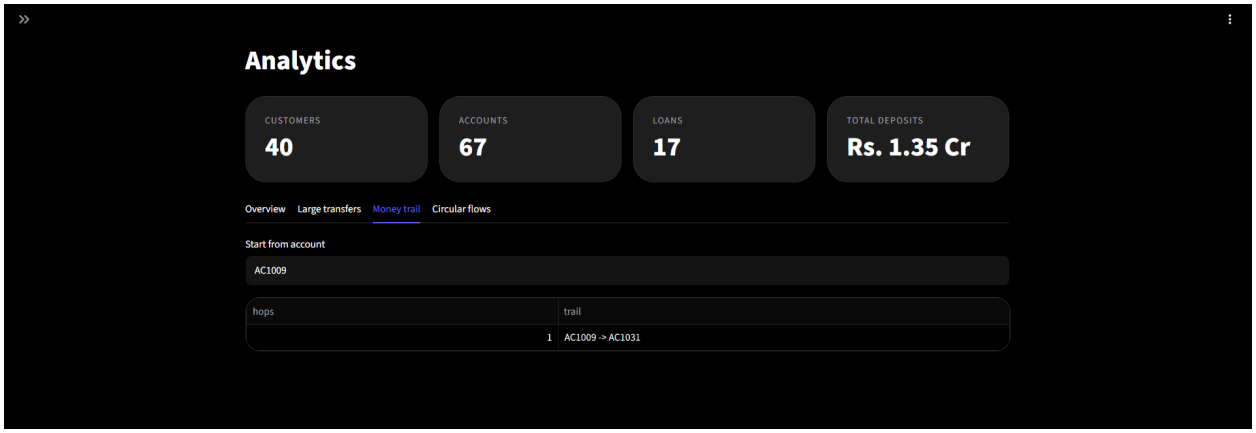
txn_id	from_account	to_account	amount	txn_date
TRX0046	AC1014	AC1062	90000	2026-09-01
TRX0047	AC1062	AC1067	88000	2026-09-02
TRX0048	AC1067	AC1065	86500	2026-09-03
TRX0049	AC1065	AC1036	85000	2026-09-04
TRX0016	AC1062	AC1057	57200	2026-02-23
TRX0039	AC1050	AC1065	56900	2026-07-04
TRX0035	AC1017	AC1026	56600	2026-07-02
TRX0009	AC1052	AC1032	56200	2026-08-19
TRX0003	AC1038	AC1032	55800	2026-04-18
TRX0036	AC1029	AC1033	54800	2026-03-17

Money trail

AC1005’s money passes through up to 3 accounts.

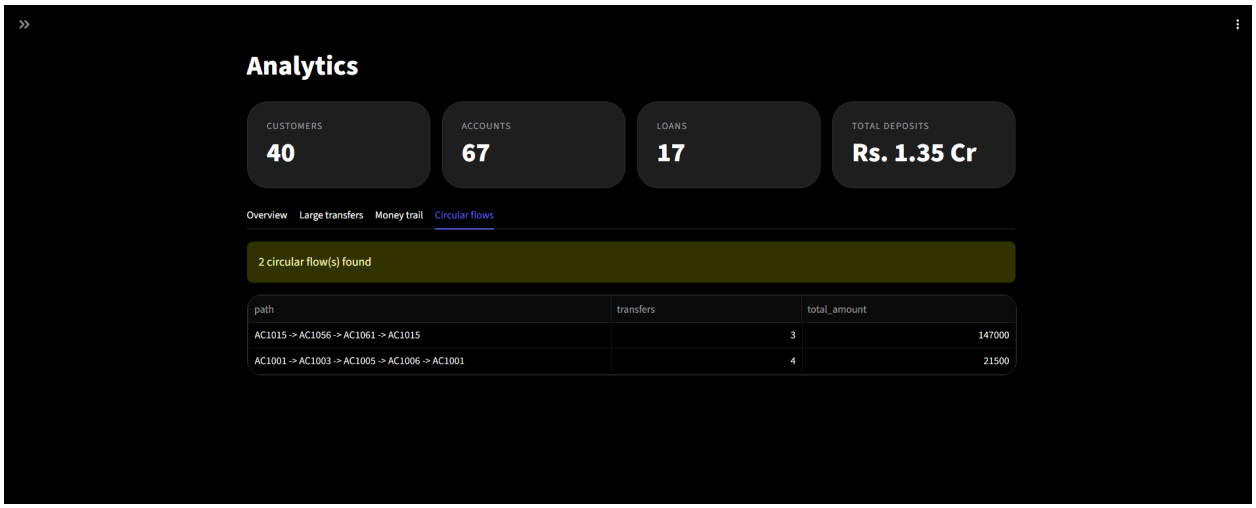


AC1009 has a single hop.



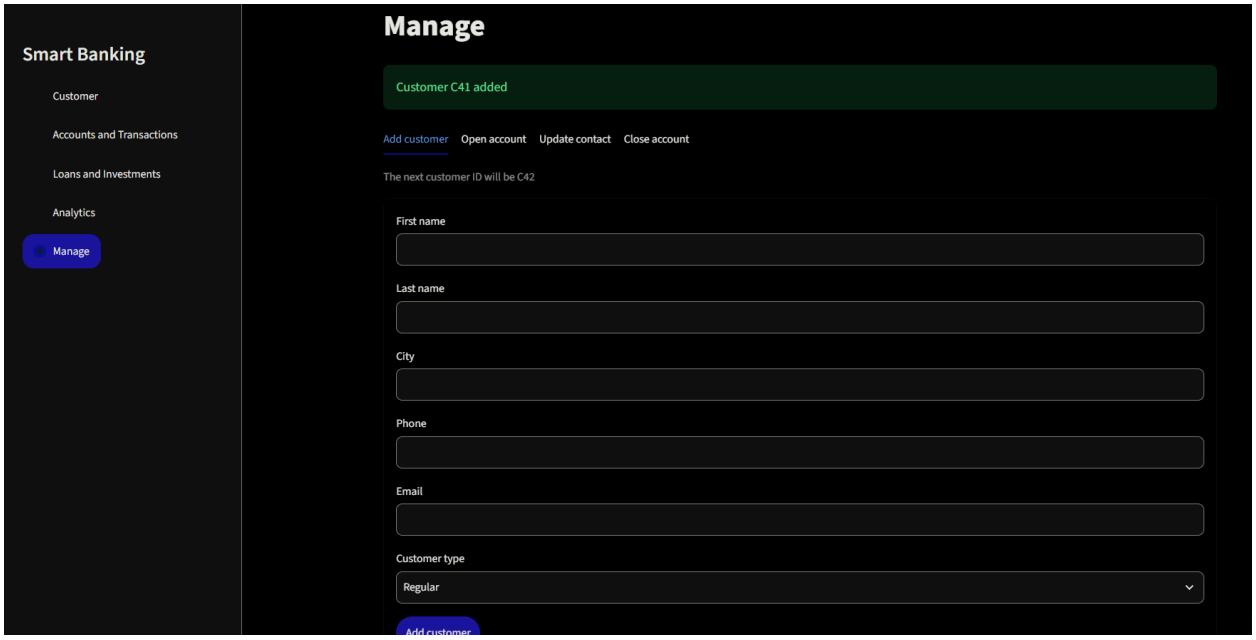
Circular flow:

2 loops found. The 147,000 loop is the suspicious one.

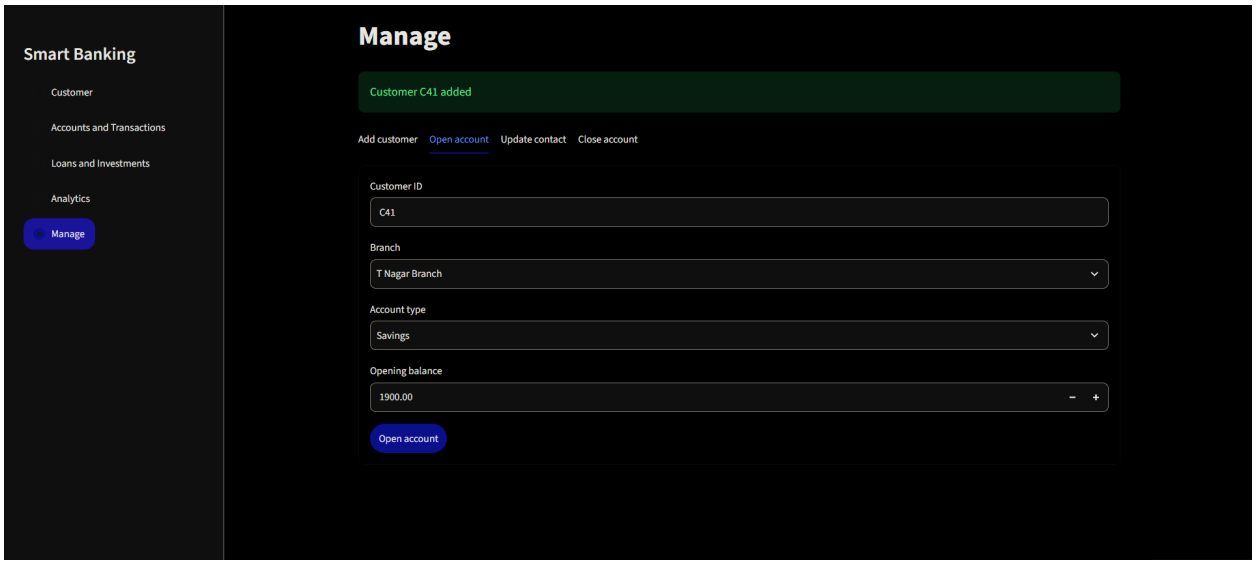


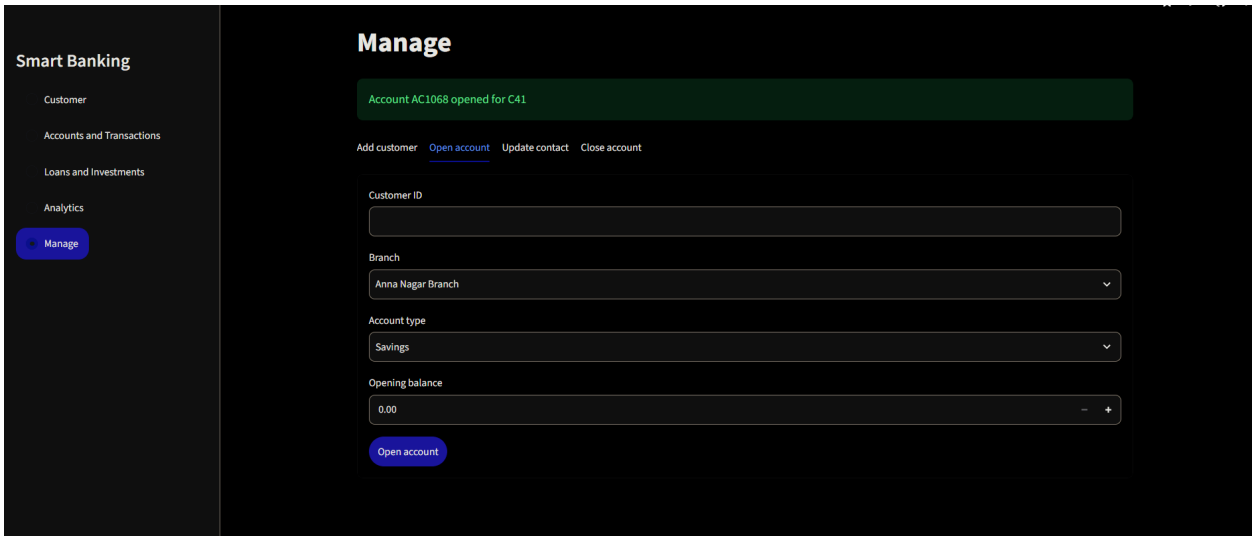
Manage:

A customer C41 is added and their details are mentioned.

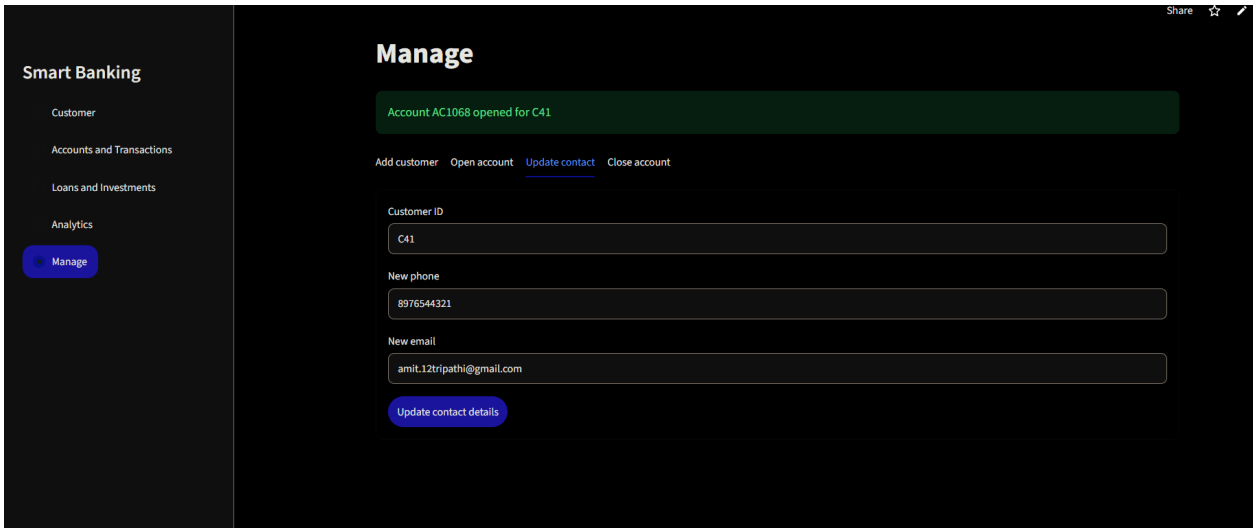


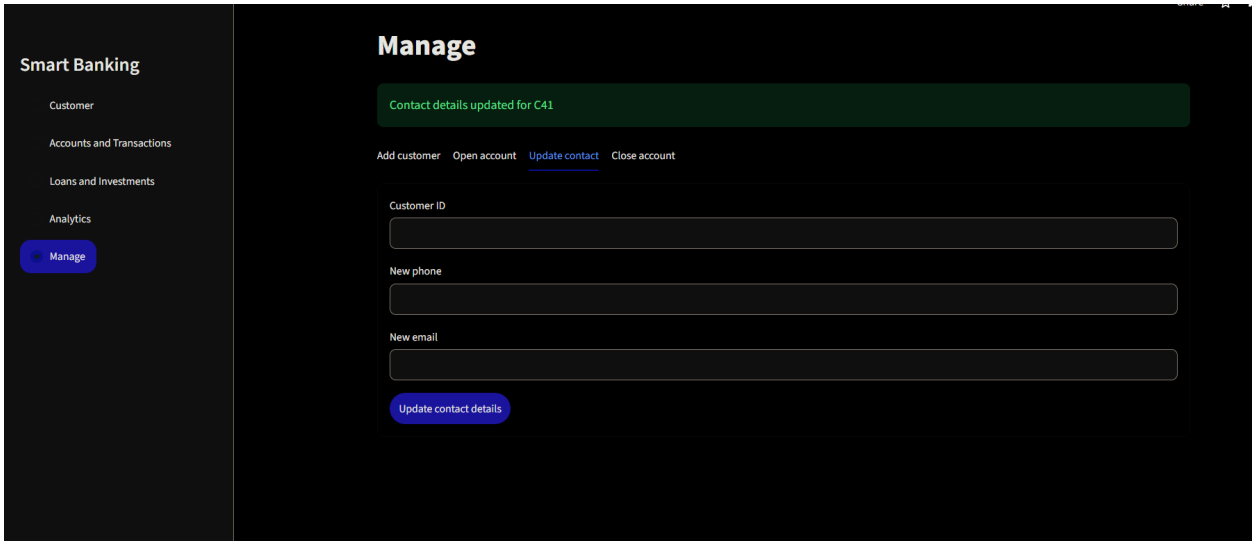
An account for customer C41 is added and the details are mentioned with initial deposit.





C41 contact details have been updated

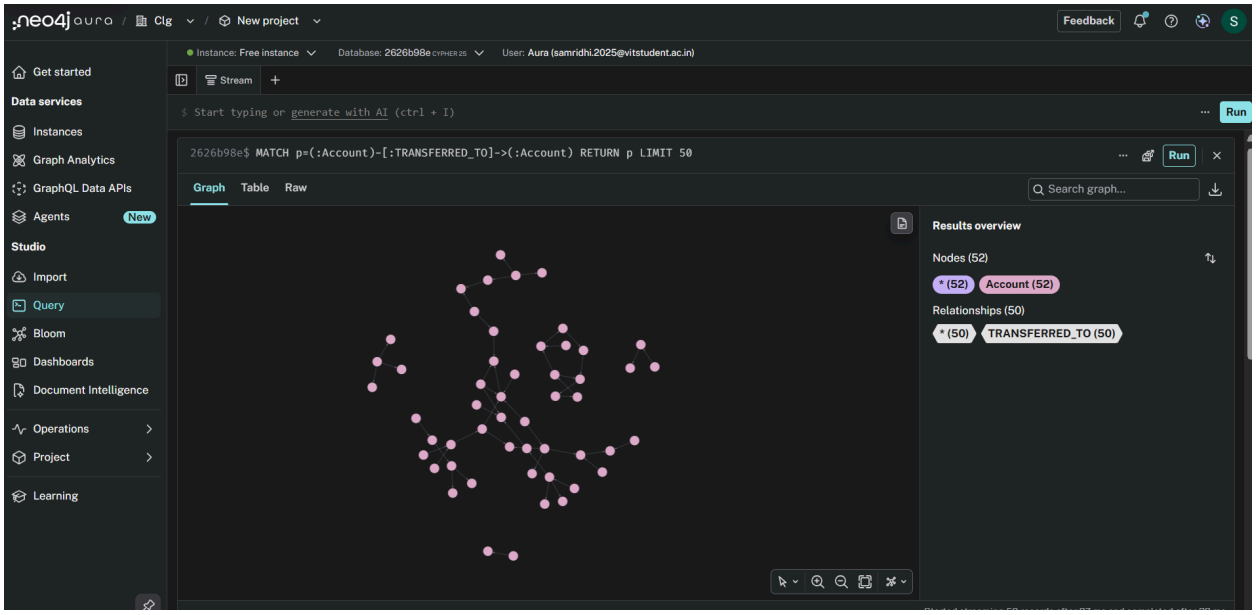




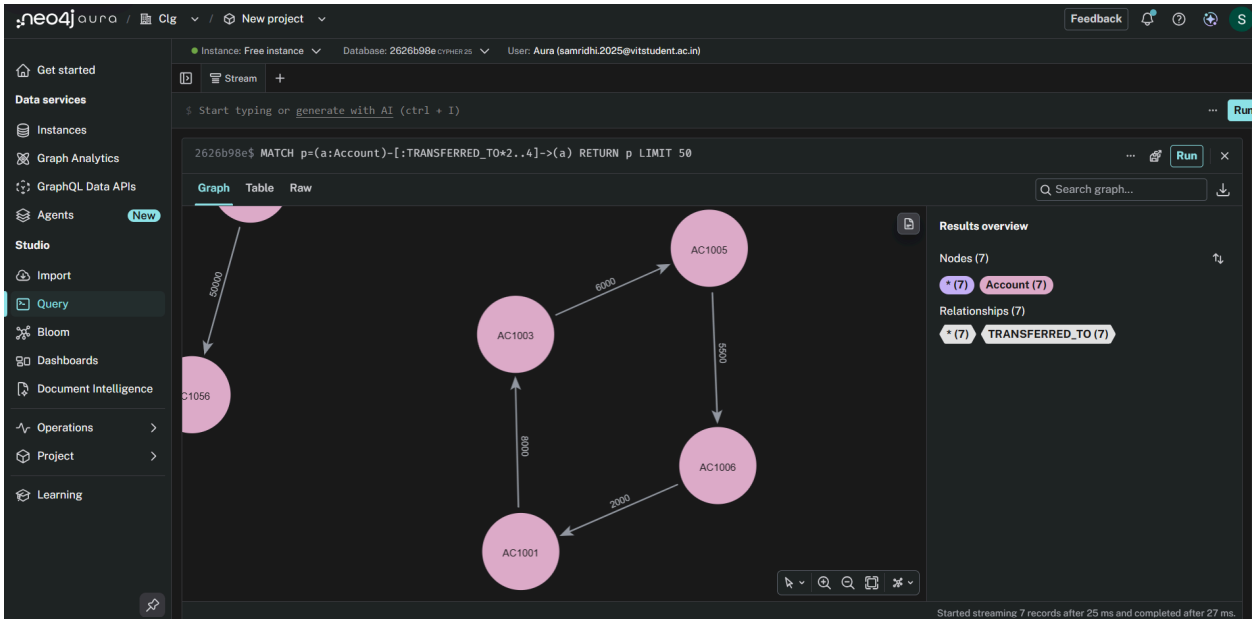
Account will be closed when the balance is zero for a customer.

NEO4j **:**

The graph views show the stored data: 52 accounts linked by 50 transfers, and a circular flow where money returns to the account it started from (AC1001 to AC1003 to AC1005 to AC1006 and back to AC1001).



It shows accounts where money returns to its starting account, for example AC1001 → AC1003 → AC1005 → AC1006 → AC1001.



Conclusion

The application stores banking data as a graph and lets users view, update and analyse it. Neo4j makes path questions such as money trails and circular flows simple, which would be much harder in a relational database.